Class 04 - Functions

Scope, argument binding, and why a function may silently use a global variable

Current path: week_01_logic -> class_04_functions

1. The case you just discovered

You renamed the function parameters from `m0` and `mf` to clearer names such as `initial_mass` and `final_mass`. However, the function body still referenced `m0` and `mf`.

The call was passing the new arguments correctly. The problem was that the function body was not using those parameters.

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): mass_ratio = m0 / mf # <-
uses outer names delta_v = Isp * g0 * np.log(mass_ratio) return delta_v
```

If the outer program contains:

```
m0 = 500000 mf = 250000
```

Python can find and use them. That is why the function appeared to ignore the new values supplied in each call.

2. What happens when a function is defined

When Python encounters a `def` statement, it does not run the calculation yet. It creates a function object and binds it to a name.

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): ...
```

The body waits until the function is called.

3. What happens when a function is called

Suppose:

```
engine_isp = 250 wet_mass = 500000 dry_mass = 100000 dv = calculate_rocket_delta_v(engine_isp,
wet_mass, dry_mass)
```

Python binds positional arguments to parameters by order:

Outer argument	Local parameter
engine_isp = 250	Isp = 250
wet_mass = 500000	initial_mass = 500000
dry_mass = 100000	final_mass = 100000
(not supplied)	g0 = 9.80665

The function now has a set of local names for this specific call.

4. Scope: where each name exists

Scope is the region where a name can be found. A function parameter is local to that call.

```
def example(initial_mass): ratio = initial_mass / 1000 return ratio
```

`initial_mass` and `ratio` are local. They do not need to exist with those names outside the function.

5. How Python looks up a name: LEGB

When Python encounters a name such as `m0`, it searches approximately in this order:

L	Local	variables in the current function
E	Enclosing	outer functions, if any
G	Global	names defined in the module/file
B	Built-in	built-in names such as len and print

In your case, because `m0` and `mf` were not local names, Python continued searching and found the global variables defined outside.

6. Incorrect vs correct version

Risky version:

```
m0 = 500000 mf = 250000 def calculate_rocket_delta_v(Isp, initial_mass, final_mass): mass_ratio = m0 / mf return Isp * 9.80665 * np.log(mass_ratio)
```

Even if you call the function with different masses, the calculation still uses the global `m0` and `mf`.

Correct version:

```
def calculate_rocket_delta_v(Isp, initial_mass, final_mass, g0=9.80665): mass_ratio = initial_mass / final_mass delta_v = Isp * g0 * np.log(mass_ratio) return delta_v
```

Now the result explicitly depends on the values received through the parameters.

7. Positional arguments and keyword arguments

With positional arguments, outer variable names do not have to match the parameter names:

```
engine_isp = 250 wet_mass = 500000 dry_mass = 100000 dv = calculate_rocket_delta_v(engine_isp,
wet_mass, dry_mass)
```

Python binds by position. With keyword arguments, however, the name on the left side of the equals sign must match the parameter:

```
dv = calculate_rocket_delta_v( Isp=engine_isp, initial_mass=wet_mass, final_mass=dry_mass )
```

The expression on the right may have any name; the left side identifies the internal parameter.

8. What return does

`return` immediately ends the current call and hands an object back to the caller.

```
result = calculate_rocket_delta_v(...)
```

If the function executes `return None`, the call ends at that point and later statements in the body are skipped. This protects calculations such as division by zero.

9. Why explicit parameters are better than hidden globals

A scientific function is easier to understand, test, and reuse when its dependencies are explicit. If it needs an initial mass and a final mass, receiving them as parameters is safer than silently reading global variables.

Practical rule: if a value may change between calls, it usually belongs in the parameter list.

10. Debugger experiment

Set a breakpoint inside the function and step in with F11. In the Variables panel, inspect Locals. You should see `Isp`, `initial_mass`, `final_mass`, and `g0`. If the body uses `m0`, notice that it is not one of those local parameters.

11. Parameter-renaming checklist

1	Change the name in the def header.
2	Change every reference inside the function body.
3	Update validations and messages.
4	Update keyword calls if they exist.
5	Update docstrings/documentation.
6	Test with very different values to expose accidental globals.

12. Final mental model

CALL -> bind arguments -> create local scope -> execute body -> name lookup if needed -> return -> back to caller

Learning Tracker

Concept	Status
Local parameters	OK
Positional arguments	OK
Keyword arguments	OK
Scope	OK
LEGB	Introduced
Accidental globals	Detected
return / return None	OK
Debugger: inspect Locals	Practice